

Capítulo 16: Pekko: Tolerancia a Fallos y Estado Duradero

1. Explicación Teórica: Jerarquía, Supervisión y Estado Encapsulado

Este capítulo profundiza en cómo Apache Pekko, el *fork* de código abierto del *toolkit* Akka, implementa los principios de **tolerancia a fallos** y **estado persistente**, los cuales son vitales para construir sistemas distribuidos robustos.

Tolerancia a Fallos por Jerarquía (Let-It-Crash)

El concepto central de la tolerancia a fallos en el Modelo de Actores es la **Supervisión**. Los actores se organizan en una **jerarquía** donde cada actor hijo es supervisado por su actor padre.

Cuando un actor falla (por una excepción inesperada), el fallo se aísla y se notifica inmediatamente a su supervisor. El supervisor decide qué hacer, aplicando una de las siguientes estrategias predefinidas:

1. **Reiniciar** (`restart`): Crea una nueva instancia del actor, limpiando su estado interno.
2. **Reanudar** (`resume`): Ignora el fallo y continúa procesando el siguiente mensaje, manteniendo el estado acumulado.
3. **Detener** (`stop`): Termina el actor de forma permanente.

Esta es la filosofía "**Let-It-Crash**" (dejar que falle), la cual asume que el fallo es un evento normal y esperado, y que es más seguro y sencillo reiniciar un componente fallido que intentar repararlo en medio de la ejecución.

Manejo del Estado Pese a los Fallos

Dado que los actores no comparten memoria mutable, su estado interno es seguro para la concurrencia. Pekko Typed ofrece dos estilos para gestionar el estado, que deben ser seleccionados en función de los requisitos del sistema:

1. **Estilo Funcional (FP Style)**: Considerado el **estilo idiomático y preferido en Scala**. El estado es **inmutable** y se pasa como un parámetro a la función que define el comportamiento (`Behavior`). Al procesar un comando, el actor produce un nuevo estado inmutable y devuelve un nuevo comportamiento (`manejarTareas(nuevoEstado)`) para el siguiente mensaje.
2. **Estado Duradero (Pekko Persistence)**: En sistemas donde el estado inmutable encapsulado no puede perderse tras un reinicio (por ejemplo, después de una falla de nodo

o una actualización del sistema), se requiere **persistencia**. Pekko Persistence (Event Sourcing o Durable State) es el módulo que permite a los actores guardar su estado de forma duradera para su **recuperación automática**.

2. Sintaxis y Estructura: Supervisión y Estado Funcional

A. Sintaxis de la Supervisión

La estrategia de supervisión se aplica al actor hijo (el supervisado) envolviendo su `Behavior` mediante `Behaviors.supervise`.

```
1  import org.apache.pekko.actor.typed.SupervisorStrategy
2  import org.apache.pekko.actor.typed.scaladsl.Behaviors
3  import scala.concurrent.duration._
4
5  // 1. Comportamiento del Actor que podría fallar
6  def miActorConEstado(): Behavior[String] =
7    Behaviors.receiveMessage { msg =>
8      if (msg == "falla")
9        throw new RuntimeException("Fallo inesperado") // Excepción
10     else
11       Behaviors.same
12   }
13
14  // 2. Definición del Supervisor (Padre) con una estrategia
15  def supervisor(): Behavior[String] =
16    Behaviors.setup { context =>
17      // Estrategia: Reiniciar el actor hijo, con un límite de 10 reintentos
18      // en 10 segundos.
19      val estrategia = SupervisorStrategy.restart
20        .withLimit(maxNrOfRetries = 10, withinTimeRange = 10.seconds)
21
22      // Spawn (crear) el actor hijo y envolverlo con la estrategia:
23      val hijo = context.spawn(
24        Behaviors.supervise(miActorConEstado())
25          .onFailure[RuntimeException](estrategia), // Aplica la
26        // estrategia solo a RuntimeException
27        name = "ActorSupervisado"
28      )
29
30      // ... el padre maneja sus propios mensajes
31      Behaviors.empty
32    }
```

Si el actor lanza una excepción (`RuntimeException`), el `SupervisorStrategy.restart` asegura que el compilador creará una nueva instancia limpia en lugar de que el sistema se detenga.

B. Implementación de Estado con Event Sourcing

Para actores cuyo estado debe ser duradero, Pekko utiliza el patrón **Event Sourcing** (Eventos Fuente), implementado con `EventSourcedBehavior`. Este enfoque garantiza que el estado pueda ser reconstruido reejecutando una secuencia inmutable de **eventos** previamente guardados.

Un `EventSourcedBehavior` requiere cuatro componentes esenciales:

1. `PersistenceId` : Un identificador único y estable para el actor persistente.
2. `emptyState` : El estado inicial cuando se crea el actor por primera vez.
3. `commandHandler` : Lógica para procesar un **Comando** y producir un **Efecto** (generalmente la persistencia de un Evento).
4. `eventHandler` : Lógica para aplicar un **Evento** persistido al estado actual, devolviendo el nuevo estado inmutable.

```
1  import org.apache.pekko.persistence.typed.scaladsl.{EventSourcedBehavior,
2  Effect}
3
4  // 1. Protocolo de Mensajes (Comandos, Eventos, Estado)
5  object EntidadCuenta {
6    // Estado Inmutable
7    final case class Estado(balance: BigDecimal)
8
9    // Comandos (lo que el actor recibe)
10   sealed trait Comando
11   final case class Depositar(monto: BigDecimal) extends Comando
12
13   // Eventos (lo que se persiste)
14   sealed trait Evento
15   final case class Depositado(monto: BigDecimal) extends Evento
16
17   // 2. Definición del Comportamiento (Handler)
18   val commandHandler: (Estado, Comando) => Effect[Evento, Estado] =
19     (estado, comando) => comando match {
20       case Depositar(monto) =>
21         // 1. Valida el comando (aquí omitido)
22         // 2. Produce el efecto: Persistir el evento Depositado
23         Effect.persist(Depositado(monto))
```

```

24     }
25
26     // 3. Aplicación del Evento (Actualización de Estado)
27     // ESTO se llama durante la recuperación y después de la persistencia.
28     val eventHandler: (Estado, Evento) => Estado =
29         (estado, evento) => evento match {
30             case Depositado(monto) =>
31                 // Crea un nuevo estado inmutable
32                 estado.copy(balance = estado.balance + monto)
33         }
34
35     def apply(id: String): Behavior[Comando] =
36         EventSourcedBehavior[Comando, Evento, Estado](
37             persistenceId = PersistenceId.ofUniqueId(id), // ID único
38             emptyState = Estado(BigDecimal(0)),           // Estado inicial
39             commandHandler = commandHandler,
40             eventHandler = eventHandler
41         )
42 }

```

El `eventHandler` es crucial porque **solo debe actualizar el estado y nunca realizar efectos secundarios**. Los efectos secundarios (como logear o enviar respuestas) se encadenan al éxito de la persistencia dentro del `commandHandler` usando `Effect.persist(...).thenRun(...)`.

3. Patrones Avanzados: Distribución y Escalamiento

Cluster Sharding (Fragmentación de Clúster)

El **Cluster Sharding** es la solución de Pekko para distribuir actores con estado persistente (llamados **entidades**) a través de múltiples nodos en un clúster. Esto es esencial cuando el número de entidades excede la capacidad de memoria de un solo nodo.

- **Abstracción de Localización:** Los actores envían mensajes a la entidad utilizando su **identificador lógico** (ej., `userId` o `PersistenceId`), sin preocuparse por la ubicación física del actor en el clúster. La extensión `ClusterSharding` se encarga de rutear el mensaje al nodo correcto a través de un actor `ShardRegion`.
- **Principio Single-Writer:** El *Sharding* asegura que solo una instancia activa de una entidad (`EventSourcedBehavior`) exista a la vez para un `PersistenceId` dado. Esto previene que múltiples instancias escriban eventos de manera interleaved, lo cual podría corromper el estado recuperado.

Recuperación Automática: Si un nodo falla, el clúster detecta el fallo. Los actores persistentes que se ejecutaban en ese nodo son rápidamente reiniciados en otro nodo sano, y

su estado es recuperado mediante la **reproducción de los eventos** persistidos en el *journal* (Event Sourcing).

Sintaxis de Cluster Sharding (Configuración Inicial) El Sharding se inicializa en cada nodo al inicio del sistema, típicamente registrando la entidad.

```
1  import org.apache.pekko.cluster.sharding.typed.scaladsl.{ClusterSharding,
    Entity, EntityRef, EntityTypeKey}
2  import org.apache.pekko.actor.typed.{ActorSystem, ActorRef}
3
4  val system: ActorSystem[Nothing] = ??? // Asumir sistema de actores
    configurado con clustering
5
6  // 1. Definición de la clave del tipo de entidad
7  val TipoCuenta = EntityTypeKey[EntidadCuenta.Comando]("CuentaBancaria")
8
9  // 2. Inicialización del Sharding (registrando el comportamiento de la
    entidad)
10 val shardRegion: ActorRef[ShardingEnvelope[EntidadCuenta.Comando]] =
11     ClusterSharding(system).init(
12         Entity(TipoCuenta) { entityContext =>
13             // entityContext.entityId es el identificador único de la cuenta
14             EntidadCuenta(entityContext.entityId)
15         }
16         // Opciones de configuración (ej. estrategia de supervisión para
            fallos de journal)
17         .withSettings(ClusterShardingSettings(system))
18     )
19
20 // 3. Obtener una referencia de entidad para interactuar
    (ClusterSharding.entityRefFor)
21 val cuentaRef: EntityRef[EntidadCuenta.Comando] =
22     ClusterSharding(system).entityRefFor(TipoCuenta, "Cuenta-ABC-123")
23
24 // Envío de un mensaje (el ShardRegion lo ruteará automáticamente)
25 cuentaRef ! EntidadCuenta.Depositar(BigDecimal(100))
```

4. Comparativa con Java y Python

Aspecto	Java (Sistemas Tradicionales)	Python (Estructuras Dinámicas)	Scala (Pekko Actor Model)
Manejo de Fallos	Bloques try-catch para	try-except (similar a Java).	Supervisión Jerárquica. El fallo

Aspecto	Java (Sistemas Tradicionales)	Python (Estructuras Dinámicas)	Scala (Pekko Actor Model)
	excepciones manejadas; interrupción del hilo para fallos inesperados.		inesperado se propaga al supervisor. Estrategia "Let-It-Crash".
Estado Compartido	Mutable. Riesgo de <i>race conditions</i> y <i>deadlocks</i> .	Mutable.	Inmutable y Encapsulado. Cada actor tiene su estado privado. El estado duradero se logra con la persistencia de eventos.
Escalamiento/Distribución	Requiere coordinación manual (ZooKeeper, Redis, etc.) para la coherencia del estado distribuido.	No aplica directamente (problemas de concurrencia de GIL).	Cluster Sharding. Distribución transparente de entidades con estado, garantizando la unicidad de la instancia activa (<code>Single-Writer Principle</code>).
Recuperación	El reinicio de la aplicación o del hilo implica la pérdida del estado en memoria.	Pérdida de estado en memoria.	Recuperación por Replay de Eventos. El actor persistente reconstruye su estado reejecutando la secuencia de eventos almacenados.

5. Best Practices (*The Scala Way*)

1. **Imponer Inmutabilidad Funcional:** Para la gestión de estado de un actor, utiliza el **estilo funcional** pasando el estado como un valor inmutable (`Estado`) en lugar de usar variables mutables (`var`) dentro de la instancia del actor.
2. **Filosofía Let-It-Crash:** Define explícitamente la **estrategia de supervisión** en el actor padre (`Behaviors.supervise`) para fallos inesperados. Las excepciones (fallos) deben ser tratadas como señales para reiniciar, no como lógica de negocio.
3. **Priorizar Event Sourcing:** Para estados que deben sobrevivir a fallos, modela la entidad usando `EventSourcedBehavior` . Recuerda que la **lógica de estado solo debe residir en**

el `eventHandler` (para ser consistente en la recuperación), y los efectos secundarios (`thenRun`) deben ejecutarse *después* de que el evento se haya persistido con éxito.

4. **Usar Cluster Sharding para Entidades:** Cuando se necesita escalar horizontalmente y mantener la unicidad de las entidades (actores con `PersistenceId`), **Cluster Sharding** es la herramienta idiomática para distribuir la carga y la resiliencia en todo el clúster.

El Modelo de Actores de Pekko es una **fábrica de resiliencia**: en lugar de intentar evitar que las máquinas fallen, asume que fallarán y diseña un sistema que puede reiniciar instantáneamente las unidades de trabajo más pequeñas (actores) bajo la estricta vigilancia de un supervisor, protegiendo la integridad del sistema completo. El estado crítico se resguarda mediante la inmutabilidad (Event Sourcing) para que, tras un fallo, la nueva instancia del actor pueda recuperar su memoria con una precisión forense.